Contrato de Servicio: Authentication Service

1. Información general

Campo	Valor
Nombre del servicio	Authentication Service
Descripción	Servicio central de identidad, autenticación, autorización y gestión de sesiones para toda la plataforma UPB-CIENTÍFICA.
Objetivo	Garantizar que solo usuarios legítimos, con los permisos correctos, accedan a los demás servicios del sistema.
Responsabilidad	Directorio de usuarios/roles/grupos/permisos propio, emisión y validación de tokens, gestión de sesiones, mapa de servicios disponibles por usuario, gestión del "Home" lógico del usuario.
Tecnología	PHP + SOAP
Protocolo de comunicación	SOAP (WSDL publicado)
Servicios consumidores	Todos: HPC Service, Distributed Storage Service, File Sync Service, Shared File Service, Photo Album Service, Streaming Service, Monitoring Service, y todos los clientes (Web, Windows, Linux, Android, iOS).
Servicios proveedores	Ninguno (es el servicio base; no depende de otro servicio de negocio, solo de su propio almacenamiento de directorio).

---

2. Funcionalidades (operaciones expuestas)

1. Login
2. Logout
3. ValidarToken
4. RefrescarToken
5. RegistrarUsuario
6. CambiarContrasena
7. RecuperarContrasena
8. ObtenerPerfil
9. ActualizarPerfil
10. ObtenerRoles
11. AsignarRol
12. RevocarRol

13. ObtenerPermisos
14. CrearGrupo
15. AgregarUsuarioAGrupo
16. RemoverUsuarioDeGrupo
17. ObtenerGruposDeUsuario
18. AdministrarUsuarios (activar/desactivar/eliminar)
19. ObtenerMapaDeServicios
20. ObtenerHomeUsuario
21. RegistrarSesion / CerrarSesionesActivas

---

3. Contrato de cada operación

> Se detallan las operaciones críticas con nivel de contrato completo. Las restantes (RefrescarToken, RecuperarContrasena, ActualizarPerfil, AsignarRol/RevocarRol, ObtenerGruposDeUsuario, AdministrarUsuarios, RegistrarSesion) siguen exactamente el mismo patrón de especificación y se detallan al final en formato resumido para no duplicar estructura.

3.1 Login

Nombre: `Login` **Descripción:** Autentica a un usuario contra el directorio propio y emite un token de sesión. **Método:** `login(usuario, contrasena, origenCliente)` **Protocolo:** SOAP (operación síncrona)

Parámetros de entrada:

Nombre	Tipo	Obligatorio	Descripción	Restricciones/Validaciones
`usuario`	string	Sí	Nombre de usuario o correo institucional	3-64 caracteres, sin espacios, formato validado por regex
`contrasena`	string	Sí	Contraseña en texto plano (viaje cifrado por transporte)	8-128 caracteres; nunca se almacena ni se loguea en texto plano
`origenCliente`	string (enum)	Sí	Identifica el tipo de cliente que solicita login	Valores permitidos: `WEB`, `WINDOWS`, `LINUX`, `ANDROID`, `IOS`

Datos de salida:

Nombre	Tipo	Descripción
`token`	string	Token de sesión firmado, con expiración
`tokenExpiracion`	datetime	Momento exacto de expiración del token
`usuarioId`	string (UUID)	Identificador único del usuario
`roles`	array<string>	Roles asignados al usuario
`mapaServicios`	array<ServicioAutorizado>	Servicios que el usuario puede consumir, con su endpoint
`homePath`	string	Ruta lógica del Home del usuario en el Storage

Posibles errores:

Código	Descripción	Posible solución
`AUTH-001`	Usuario inexistente	Verificar nombre de usuario; sugerir registro si aplica
`AUTH-002`	Contraseña incorrecta	Reintentar; tras N intentos, bloqueo temporal
`AUTH-003`	Usuario deshabilitado/bloqueado	Contactar a administrador
`AUTH-004`	Origen de cliente no soportado	Verificar valor de `origenCliente`
`AUTH-005`	Servicio de directorio no disponible	Reintentar con backoff; alertar a Monitoring

Reglas de negocio:

- Tras 5 intentos fallidos consecutivos en 15 minutos, la cuenta se bloquea temporalmente (15 min) — previene fuerza bruta.
- Un login exitoso invalida tokens anteriores del mismo `origenCliente` si la política de "sesión única por dispositivo" está activa para ese usuario (configurable por rol).
- El `mapaServicios` se calcula dinámicamente según los roles/permisos del usuario en el momento del login, no se cachea permanentemente.

Seguridad:

- Requiere autenticación: No (es la operación que la otorga).
- Requiere autorización: No.
- Roles permitidos: N/A (público, sujeto a rate limiting).

- Permisos requeridos: Ninguno.

Dependencias: Ninguna (opera sobre el directorio propio del servicio).

Flujo interno:

1. Capa de Presentación (SOAP) recibe la solicitud y valida formato de entrada.
2. Capa de Aplicación invoca el caso de uso "AutenticarUsuario".
3. Capa de Dominio verifica reglas: existencia de usuario, estado de cuenta, política de bloqueo.
4. Capa de Persistencia consulta el directorio (hash de contraseña con algoritmo de hash lento, ej. bcrypt/argon2).
5. Si es válido, el Dominio genera el token (firmado, con expiración) y calcula roles/permisos/mapa de servicios.
6. Persistencia registra la sesión activa (para permitir revocación posterior).
7. Presentación devuelve la respuesta SOAP con token y metadatos.

3.2 ValidarToken

Nombre: `ValidarToken` **Descripción:** Verifica si un token es válido, no ha expirado y no ha sido revocado; devuelve los permisos asociados. **Método:** `validarToken(token)` **Protocolo:** SOAP (síncrono, alta frecuencia de invocación por parte de otros servicios)

Parámetros de entrada:

Nombre	Tipo	Obligatorio	Descripción	Restricciones/Validaciones
`token`	string	Sí	Token emitido por `Login`	Formato de firma válido antes de consultar persistencia

Datos de salida:

Nombre	Tipo	Descripción
`esValido`	boolean	Indica si el token es válido
`usuarioId`	string (UUID)	Usuario propietario del token (si es válido)
`roles`	array<string>	Roles del usuario (si es válido)
`permisos`	array<string>	Permisos efectivos del usuario (si es válido)
`expiracion`	datetime	Expiración restante del token

Posibles errores:

Código	Descripción	Posible solución
`AUTH-101`	Token expirado	Solicitar `RefrescarToken` o nuevo `Login`
`AUTH-102`	Token revocado (logout previo)	Requerir nuevo `Login`
`AUTH-103`	Firma de token inválida (manipulado)	Rechazar y registrar evento de seguridad

Reglas de negocio:

- Esta operación debe resolverse mayormente **sin round-trip a persistencia** si el token es JWT autocontenido y firmado, verificando solo la firma y expiración localmente; solo se consulta la lista de revocación si el token no ha expirado (para permitir `Logout` inmediato).
- Todo intento con firma inválida se registra como evento de seguridad para Monitoring.

Seguridad: No requiere autenticación adicional (el propio token es la credencial). No requiere autorización adicional para invocarla, pero solo servicios registrados internamente deberían consumirla (red interna, no expuesta a clientes finales directamente salvo para verificación de sesión).

Dependencias: Ninguna externa.

Flujo interno:

1. Presentación recibe token.
2. Aplicación decodifica y verifica firma/expiración sin tocar base de datos.
3. Si la firma y expiración son válidas, Dominio consulta lista de revocación (solo si existe cache/registro de logout).
4. Si todo es válido, se devuelven roles/permisos ya embebidos en el token o recalculados si la política exige "permisos siempre frescos".

3.3 Logout

Nombre: `Logout` **Descripción:** Invalida la sesión activa asociada a un token. **Protocolo:** SOAP

Parámetros de entrada: `token` (string, obligatorio).

Datos de salida: `confirmacion` (boolean).

Posibles errores: `AUTH-101` (token ya expirado, se ignora y se confirma logout de todas formas), `AUTH-102` (token ya revocado).

Reglas de negocio: El token se agrega a una lista de revocación válida hasta su expiración natural (no se puede "desfirmar" un JWT, solo revocarlo explícitamente).

Seguridad: Requiere el token válido de la sesión a cerrar.

Dependencias: Ninguna.

Flujo interno: Presentación recibe token → Aplicación valida que exista sesión activa → Dominio marca el token como revocado → Persistencia guarda la revocación → confirmación al cliente.

3.4 RegistrarUsuario

Nombre: `RegistrarUsuario` **Protocolo:** SOAP

Parámetros de entrada:

Nombre	Tipo	Obligatorio	Descripción	Restricciones
`nombreUsuario`	string	Sí	Identificador único de login	Único en el directorio, 3-64 caracteres
`correo`	string	Sí	Correo institucional	Formato de correo válido, único
`contrasena`	string	Sí	Contraseña inicial	Mínimo 8 caracteres, con política de complejidad
`nombreCompleto`	string	Sí	Nombre para mostrar	1-128 caracteres
`rolInicial`	string (enum)	No	Rol con el que se crea la cuenta	Por defecto: `INVESTIGADOR`

Datos de salida: `usuarioId`, `homePath` (Home recién creado), `confirmacion`.

Posibles errores:

Código	Descripción	Solución
`AUTH-201`	Nombre de usuario ya existe	Elegir otro
`AUTH-202`	Correo ya registrado	Recuperar cuenta existente
`AUTH-203`	Contraseña no cumple política	Ajustar contraseña

Reglas de negocio: Al crear el usuario se debe **provisionar automáticamente su Home** (evento emitido para que Distributed Storage Service lo escuche y cree el directorio correspondiente — ver sección 5, Eventos). El registro no otorga acceso hasta confirmación (si se define verificación por correo, queda fuera del alcance mínimo pero se deja como extensión).

Seguridad: Puede requerir autorización de administrador si el registro es "cerrado" (no autoregistro público) — configurable por política institucional.

Dependencias: Emite evento consumido por Distributed Storage Service.

Flujo interno: Presentación valida formato → Aplicación verifica unicidad → Dominio aplica política de contraseña y rol por defecto → Persistencia crea el registro → Dominio emite evento `UsuarioRegistrado` → respuesta al cliente.

3.5 ObtenerPerfil / ObtenerRoles / ObtenerPermisos

Protocolo: SOAP. **Entrada:** `token` o `usuarioId` + `token` del solicitante (para validar que puede consultar ese perfil). **Salida:** datos del perfil / lista de roles / lista de permisos efectivos. **Errores:** `AUTH-101/102/103` (token inválido), `AUTH-301` (permiso insuficiente para consultar perfil de otro usuario). **Regla de negocio:** un usuario siempre puede consultar su propio perfil; consultar el de otro requiere rol `ADMIN` o permiso explícito `usuarios:leer`.

3.6 CrearGrupo / AgregarUsuarioAGrupo / RemoverUsuarioDeGrupo

Protocolo: SOAP. **Entrada (CrearGrupo):** `nombreGrupo`, `descripcion`, `token` del solicitante. **Salida:** `grupoId`. **Errores:** `AUTH-401` (grupo ya existe), `AUTH-301` (permiso insuficiente — requiere rol `ADMIN` o permiso `grupos:administrar`). **Regla de negocio:** los permisos de un grupo se heredan por todos sus miembros; un usuario puede pertenecer a múltiples grupos y sus permisos efectivos son la unión de todos.

---

4. Modelo de datos

Usuario

Atributo	Tipo	Descripción	Restricciones
`id`	UUID	Identificador único	PK, inmutable
`nombreUsuario`	string	Login	Único, 3-64 caracteres
`correo`	string	Correo institucional	Único, formato válido
`contrasenaHash`	string	Hash de la contraseña	Nunca se expone en ningún contrato de salida
`nombreCompleto`	string	Nombre para mostrar	1-128 caracteres
`estado`	enum	`ACTIVO`, `BLOQUEADO`, `DESHABILITADO`	—
`fechaCreacion`	datetime	Auditoría	Inmutable
`homePath`	string	Ruta lógica del Home	Asignada al crear el usuario

Rol

Atributo	Tipo	Descripción
`id`	UUID	Identificador
`nombre`	string	Ej. `ADMIN`, `INVESTIGADOR`, `OPERADOR`
`descripcion`	string	—
`permisos`	array<Permiso>	Permisos asociados al rol

Grupo

Atributo	Tipo	Descripción
`id`	UUID	Identificador
`nombre`	string	Único
`miembros`	array<UUID>	Usuarios pertenecientes
`permisos`	array<Permiso>	Permisos heredados por los miembros

Permiso

Atributo	Tipo	Descripción
`codigo`	string	Ej. `archivos:leer`, `hpc:enviar-trabajo`
`descripcion`	string	—

Token

Atributo	Tipo	Descripción
`valor`	string	Token firmado
`usuarioId`	UUID	Propietario
`origenCliente`	enum	Cliente que lo emitió
`fechaEmision`	datetime	—
`fechaExpiracion`	datetime	—
`revocado`	boolean	Si fue invalidado por Logout

ServicioAutorizado

Atributo	Tipo	Descripción
`nombreServicio`	string	Ej. `HPC Service`
`endpoint`	string	URL/dirección del servicio
`protocolo`	enum	`SOAP`, `RMI`, `gRPC`, `HTTP`

---

5. Eventos que genera el servicio

Evento	Disparado cuando	Consumidores típicos
`UsuarioRegistrado`	Se completa `RegistrarUsuario`	Distributed Storage Service (crear Home)
`UsuarioAutenticado`	Login exitoso	Monitoring Service (métricas de acceso)
`SesionCerrada`	Logout	Monitoring Service
`IntentoFallidoDeLogin`	Contraseña incorrecta o cuenta bloqueada	Monitoring Service (alertas de seguridad)
`RolAsignado` / `RolRevocado`	Cambios de rol	Servicios que cachean permisos localmente (para invalidar cache)
`UsuarioDeshabilitado`	AdministrarUsuarios	Todos los servicios (para forzar cierre de sesiones activas)

---

6. Integración

- **Quién consume este contrato:** absolutamente todos los servicios de negocio (para `ValidarToken`) y todos los clientes (para `Login`, `Logout`, `ObtenerPerfil`, etc.).
- **Quién responde:** únicamente el propio Authentication Service; no delega en otros servicios.
- **Protocolo:** SOAP exclusivamente, vía WSDL versionado.
- **Momento del flujo en que participa:** siempre es el primer paso de cualquier flujo de usuario (login) y un paso intermedio obligatorio en cada operación de cualquier otro servicio (validación de token/permiso).

---

7. Consideraciones de escalabilidad

- **Qué puede crecer:** `ValidarToken` es, por lejos, la operación de mayor volumen — la invoca cada operación de cada servicio. Debe optimizarse para no depender de una consulta a base de datos por cada llamada (tokens autocontenidos y firmados, con revocación consultada solo cuando aplica).

- **Cuellos de botella posibles:** si el directorio de usuarios crece mucho, las consultas de pertenencia a grupos (permisos heredados) pueden volverse costosas — conviene precalcular permisos efectivos al emitir el token en lugar de recalcularlos en cada validación.
- **Mejoras futuras:** cache distribuida de validación de tokens; réplica de solo lectura del directorio para consultas de perfil/roles que no modifican estado.

---

8. Casos límite

Operación	Caso límite	Comportamiento esperado
`Login`	Dos logins simultáneos del mismo usuario desde distintos clientes	Ambos se permiten salvo que la política de "sesión única" esté activa; en ese caso el segundo invalida el token del primero
`Login`	Ataque de fuerza bruta (muchos intentos en poco tiempo)	Bloqueo temporal tras N intentos; no revelar si el error fue usuario o contraseña (evitar enumeración de usuarios)
`ValidarToken`	Token válido pero el usuario fue deshabilitado después de emitirlo	Debe rechazarse — requiere verificar estado de usuario, no solo firma del token
`RegistrarUsuario`	Dos registros simultáneos con el mismo nombre de usuario	Debe existir restricción de unicidad a nivel de persistencia (no solo validación en Aplicación) para evitar condición de carrera
`CrearGrupo`	Grupo eliminado mientras un usuario está siendo agregado a él	La operación debe fallar de forma controlada (`AUTH-401` o equivalente), no dejar estado inconsistente
`Logout`	Logout de un token que ya expiró naturalmente	Se responde `confirmacion: true` igualmente — el resultado final (sesión inválida) es el mismo

Resumen de operaciones no detalladas en extenso (mismo patrón de contrato)

Operación	Entrada clave	Salida clave	Error principal	Seguridad
`RefrescarToken`	`token` vigente	nuevo `token` + expiración	`AUTH-101` si ya expiró (requiere login)	Token válido
`RecuperarContrasena`	`correo`	confirmación de envío de instrucciones	`AUTH-001` si no existe (respuesta genérica por seguridad)	Público, con rate limiting
`ActualizarPerfil`	`token`, campos a actualizar	perfil actualizado	`AUTH-301` si intenta modificar campos restringidos	Requiere token propio
`AsignarRol` / `RevocarRol`	`usuarioId`, `rol`, `token` admin	confirmación	`AUTH-301` permiso insuficiente	Rol `ADMIN` o perm `roles:administ`
`ObtenerGruposDeUsuario`	`usuarioId`, `token`	lista de grupos	`AUTH-301` si consulta usuario ajeno sin permiso	Propio o `ADMIN`
`AdministrarUsuarios`	`usuarioId`, `accion` (activar/desactivar/eliminar), `token` admin	confirmación	`AUTH-301`	Rol `ADMIN`
`ObtenerMapaDeServicios`	`token`	lista de `ServicioAutorizado`	`AUTH-101/102/103`	Token válido
`ObtenerHomeUsuario`	`token`	`homePath`	`AUTH-101/102/103`	Token propio
`RegistrarSesion` / `CerrarSesionesActivas`	`usuarioId`, `token` admin (para forzar cierre remoto)	confirmación	`AUTH-301`	Propio o `ADMIN`

Contrato listo para que cualquier equipo implemente Authentication Service en PHP/SOAP sin necesidad de consultar el código de otros servicios. Siguiente contrato en la secuencia: Distributed Storage Service.